

✓ Phishing URL Triage Tool

Nic Recasens jr38088@georgiasouthern.edu

Nigel Smith ns15468@georgiasouthern.edu

Georgia Southern University Department of Computer Science
CSCI 7090 - Data Science and Machine Learning - Spring 2026

✓ Overview

Triage tool that combines our two ML pipelines: SVM embeddings and K-Means classification. A suspicious URL is input, SVM returns likelihood of impersonation, and K-Means classifies based on closest fit attack pattern. Runs directly in the notebook off of pipeline data.

It is important to note that this does not serve the purpose(s) of a standalone detector; we are assuming the URL it is given as input is already under suspicion. This is for demonstration purposes.

A complete implementation would incorporate as a step 0 a binary classification of phishing/legitimate.

```
import os
import re
from pathlib import Path
from urllib.parse import urlparse

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split

import warnings
warnings.filterwarnings('ignore', category=UserWarning)
```

```
if not os.path.exists('DS_ML_Project_ColabIntegration'):
    !git clone https://github.com/ns15468-gasou/DS_ML_Project_ColabIntegration.git

os.chdir('DS_ML_Project_ColabIntegration/Project/notebooks')
print("Working Dir: ", os.getcwd())
```

```
Working Dir: /home/nic/DS_ML_Project_ColabIntegration/Project/notebooks/DS_ML_Project_Col
```

```
BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data"
```

```
PHISHTANK_FILE = DATA_DIR / "raw_input" / "verified_online.csv"
TOP5_PHISH_FILE = DATA_DIR / "summarized_output" / "top5_per_phish.csv"
URL_MAP_FILE = DATA_DIR / "processed_input" / "url_cleaning_map_phishtank.csv"
```

```
RANDOM_STATE = 777
TEST_SIZE = 0.2
```

✓ Data Loading

```
def load_data(filepath):
    df = pd.read_csv(filepath)
    print(f"Loaded {len(df):,} phishing URLs")
    return df

def load_cleaning_map(filepath):
    map_df = pd.read_csv(filepath)
    deduped = map_df.drop_duplicates(subset='cleaned_url', keep='first')
    return dict(zip(deduped['cleaned_url'], deduped['url']))
```

```
df_sim = load_data(TOP5_PHISH_FILE)
url_map = load_cleaning_map(URL_MAP_FILE)
```

```
Loaded 54,896 phishing URLs
```

```
def restore_url(cleaned: str, url_map: dict) -> str:
    """Restore a cleaned URL to its pre-cleaning form via the cleaning map,
    then prepend a scheme so it is parseable by urlparse."""
    restored = url_map.get(cleaned, cleaned)
    if not re.match(r'^https?://', restored):
        restored = 'http://' + restored
    return restored

def extract_features(df: pd.DataFrame, url_map: dict,
                    url_col: str = "item") -> pd.DataFrame:
    """Compute structural URL features from a column of (cleaned) URL strings.

    Uses the cleaning map to restore URLs before parsing with urlparse,
    giving accurate host / path extraction.
    """
    urls = df[url_col].apply(lambda u: restore_url(u, url_map))
    hosts = urls.apply(lambda u: urlparse(u).netloc.lower())
    paths = urls.apply(lambda u: urlparse(u).path

    feats = pd.DataFrame(index=df.index)
    feats['full_length'] = df[url_col].str.len()
    feats['host_length'] = hosts.str.len()
```

```

feats['path_depth']      = paths.apply(lambda p: p.strip('/').count('/') + 1 if p.sti
feats['dot_count']       = df[url_col].str.count(r'\.')
feats['hyphen_count']    = df[url_col].str.count('-')
feats['digit_count']     = df[url_col].str.count(r'\d')
feats['digit_ratio']     = feats['digit_count'] / feats['full_length']
feats['subdomain_depth'] = hosts.str.count(r'\.')
return feats

```

```

def create_labels(df: pd.DataFrame, quantile: float = 0.75) -> tuple: # 75 percentile be:
    """Create binary impersonation label from score_1 using the given quantile threshold
    threshold = df['score_1'].quantile(quantile)
    df = df.copy()
    df['is_impersonating'] = (df['score_1'] >= threshold).astype(int)
    label_counts = df['is_impersonating'].value_counts()

    print(f"Impersonation threshold ({quantile:.0%} percentile): {threshold:.4f}")
    print(f"\nClass distribution:")
    print(f"  0 (not impersonating): {label_counts.get(0, 0):,}")
    print(f"  1 (impersonating):      {label_counts.get(1, 0):,}")
    print(f"  Ratio: {label_counts.get(1, 0) / len(df):.2%}")

    return df, threshold, label_counts

```

```

df_sim, threshold, _ = create_labels(df_sim)
features = extract_features(df_sim, url_map)
feature_names = features.columns.tolist()

```

Impersonation threshold (75% percentile): 0.7460

Class distribution:

0 (not impersonating):	41,172
1 (impersonating):	13,724
Ratio:	25.00%

✓ SVM Model

Uses parameters $C=3.1489$ and $\gamma = 0.7114$ as found in NB2.0 Retuning is skipped for brevity.

```

X = features.values
y = df_sim['is_impersonating'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=TEST_SIZE, random_st

svm_model = Pipeline([('scaler', StandardScaler()), ('svm', SVC(kernel='rbf', C=3.1489, {
svm_model.fit(X_train, y_train)
print("SVM trained.")

```

SVM trained.

✓ Gradient Boosting — Impersonation Classifier (SVM Benchmark)

GBT trained on the same is_impersonating label and feature set as the SVM for direct AUC comparison.

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, classification_report

gbt_imp_model = GradientBoostingClassifier(n_estimators=100, random_state=RANDOM_STATE)
gbt_imp_model.fit(X_train, y_train)
y_prob_gbt = gbt_imp_model.predict_proba(X_test)[: , 1]
y_pred_gbt = gbt_imp_model.predict(X_test)

print(f"GBT AUC: {roc_auc_score(y_test, y_prob_gbt):.3f}")
print(f"SVM AUC: 0.809")
print(classification_report(y_test, y_pred_gbt, target_names=['Not Impersonating', 'Impe
```

GBT AUC: 0.898

SVM AUC: 0.809

	precision	recall	f1-score	support
Not Impersonating	0.86	0.97	0.91	8235
Impersonating	0.86	0.53	0.66	2745
accuracy			0.86	10980
macro avg	0.86	0.75	0.79	10980
weighted avg	0.86	0.86	0.85	10980

✓ K-Means Model

Retrained on "other" PhishTank dataset, replicates clusters from NB2.1.

```
phish = pd.read_csv(PHISHTANK_FILE)

phish['host'] = phish['url'].apply(lambda x: urlparse(x).netloc.lower())
phish['full_length'] = phish['url'].str.len()
phish['host_length'] = phish['host'].str.len()
phish['path'] = phish['url'].apply(lambda x: urlparse(x).path)
phish['path_depth'] = phish['path'].apply(lambda p:
p.strip('/').count('/') + 1 if p.strip('/') else 0)
phish['dot_count'] = phish['url'].str.count(r'\.')
phish['hyphen_count'] = phish['url'].str.count('-')
phish['digit_count'] = phish['url'].str.count(r'\d')
phish['digit_ratio'] = phish['digit_count'] / phish['full_length']
phish['subdomain_depth'] = phish['host'].str.count(r'\.')

other = phish[phish['target'] == 'Other'].copy()

cluster_features = ['host_length', 'path_depth', 'dot_count', 'hyphen_count', 'digit_rat:
X_cluster = other[cluster_features].dropna()

scaler_km = StandardScaler()
```

```
X_scaled = scaler_km.fit_transform(X_cluster)
```

```
km_model = KMeans(n_clusters=4, random_state=777, n_init=10)
km_model.fit(X_scaled)
print("K-Means trained.")
```

K-Means trained.

```
cluster_names = {
    0: 'Platform Abuse (Simple)',
    1: 'Platform Abuse (Complex)',
    2: 'Token / Long URL Abuse',
    3: 'Shortener / Redirector'
}
```

✓ Triage Function

`classify_url()` takes a raw URL string, extracts the same features as used in previous models, then returns impersonation probability from the SVM and attack pattern cluster from K-Means.

```
def classify_url(url):
    if not re.match(r'^https?://', url):
        url = 'http://' + url

    parsed = urlparse(url)
    host = parsed.netloc.lower()
    path = parsed.path

    full_length      = len(url)
    host_length      = len(host)
    path_depth       = path.strip('/').count('/') + 1 if path.strip('/') else 0
    dot_count        = url.count('.')
    hyphen_count     = url.count('-')
    digit_count      = sum(c.isdigit() for c in url)
    digit_ratio      = digit_count / full_length
    subdomain_depth  = host.count('.')

    feat_vector = np.array([[full_length, host_length, path_depth,
                             dot_count, hyphen_count, digit_count, digit_ratio, subdomain_depth]])

    imp_prob = svm_model.predict_proba(feat_vector)[0][1]
    imp_label = 'LIKELY IMPERSONATING' if imp_prob >= 0.5 else 'LIKELY NOT IMPERSONATING'

    km_input = np.array([[host_length, path_depth, dot_count, hyphen_count, digit_ratio]])

    km_scaled = scaler_km.transform(km_input)
    cluster = km_model.predict(km_scaled)[0]
    cluster_name = cluster_names[cluster]

    print(f"URL: {url} | Impersonation Probability: {imp_prob} | Cluster: {cluster_name}")
```

```
print(f"Impersonation:    {imp_prob:.1%} – {imp_label}")
print(f"Attack Pattern:   Cluster {cluster} – {cluster_name}")
```

▼ Demo

URL is passed to 'classify_url()' for triage tool. Again, all inputs passed here are already marked as suspicious.

```
classify_url("google.com")
classify_url("googl.io")
classify_url("sites.google.com/view/paypal-secure-login/home")
classify_url("https://qrco.de/bfyD0b")
classify_url("https://docs.google.com/presentation/d/e/2PACX-1vS96MJlExf8/pub?start=false")
```

```
URL:          http://google.com
Impersonation: 49.0% – LIKELY NOT IMPERSONATING
Attack Pattern: Cluster 3 – Shortener / Redirector
URL:          http://googl.io
Impersonation: 47.0% – LIKELY NOT IMPERSONATING
Attack Pattern: Cluster 3 – Shortener / Redirector
URL:          http://sites.google.com/view/paypal-secure-login/home
Impersonation: 47.4% – LIKELY NOT IMPERSONATING
Attack Pattern: Cluster 2 – Token / Long URL Abuse
URL:          https://qrco.de/bfyD0b
Impersonation: 61.9% – LIKELY IMPERSONATING
Attack Pattern: Cluster 3 – Shortener / Redirector
URL:          https://docs.google.com/presentation/d/e/2PACX-1vS96MJlExf8/pub?start=false
Impersonation: 3.6% – LIKELY NOT IMPERSONATING
Attack Pattern: Cluster 2 – Token / Long URL Abuse
```

```
test_urls = [
    "google.com",
    "googl.io",
    "sites.google.com/view/paypal-secure-login/home",
    "https://qrco.de/bfyD0b",
    "https://docs.google.com/presentation/d/e/2PACX-1vS96MJlExf8/pub?start=false"
]
```

```
rows = []
for raw in test_urls:
    url = raw if re.match(r'^https?://', raw) else 'http://' + raw
    parsed = urlparse(url)
    host = parsed.netloc.lower()
    path = parsed.path
    full_length = len(url)
    host_length = len(host)
    path_depth = path.strip('/').count('/') + 1 if path.strip('/') else 0
    dot_count = url.count('.')
    hyphen_count = url.count('-')
    digit_count = sum(c.isdigit() for c in url)
    digit_ratio = digit_count / full_length
    subdomain_depth = host.count('.')
    rows.append([url, host, path, path_depth, dot_count, hyphen_count, digit_ratio, subdomain_depth])
```

```

feat_vector = np.array([[full_length, host_length, path_depth, dot_count, hyphen_cour
imp_prob = svm_model.predict_proba(feet_vector)[0][1]
km_scaled = scaler_km.transform(np.array([[host_length, path_depth, dot_count, hyper
cluster = km_model.predict(km_scaled)[0]
rows.append({
    'URL': raw,
    'Impersonation': f'{imp_prob:.1%}',
    'Verdict': 'Likely Impersonating' if imp_prob >= 0.5 else 'Likely Not Impersonat:
    'Attack Pattern': cluster_names[cluster]
})

```

```
display(pd.DataFrame(rows).style.hide(axis='index'))
```

	URL	Impersonation	Verdict	Attack Pattern
	google.com	49.0%	Likely Not Impersonating	Shortener / Redirector
	googl.io	47.0%	Likely Not Impersonating	Shortener / Redirector
	sites.google.com/view/paupal-secure-login/home	47.1%	Likely Not	Token / Long

References and Data Sources

- [1] Gabriela Brezeanu, Alexandru Archip, and Codrut-Georgian Artene. Phish fighter: Self updating machine learning shield against phishing kits based on HTML code analysis. 13:4460-4486.
- [2] Bibhu Dash and Meraj Farheen Ansari. An effective cybersecurity awareness training model: First defense of an organizational security strategy. International Research Journal of Engineering and Technology, 9:2395-0056, 04 2022.
- [3] Dawn M. Sarno, Maggie W. Harris, and Jeffrey Black. Which phish is captured in the net? understanding phishing susceptibility and individual differences. 37(4):789-803. [eprint:onlinelibrary.wiley.com/doi/pdf/10.1002/acp.4075](https://onlinelibrary.wiley.com/doi/pdf/10.1002/acp.4075).
- [4] Giuseppe Desolda, Francesco Greco, and Luca Vigano. APOLLO: A GPT-based tool to detect phishing emails and generate explanations that warn users. 9(4):EICS003:1-EICS003:33.
- [5] Nadjate Saidani, Kamel Adi, and Mohand Saïd Allili. A semantic-based classification approach for an enhanced spam detection. Computers & Security, 94, 2020-07.
- [6] Sonowal and Gunikhan. Detecting phishing sms based on multiple correlation algorithms. SN Computer Science, 1, 11 2020.
- [7] Panpan Zhang, Jing Ya, Tingwen Liu, Quangang Li, Jinqiao Shi, and Zhaojun Gu. imcircle: Automatic mining of indicators of compromise from the web. In 2019 IEEE symposium on Computers and Communications (ISCC), 2019.
- [8] Rasha Zieni, Luisa Massari, and Maria Carla Calzarossa. Phishing or not phishing? a survey on the detection of phishing websites. 11:18499-18519.

[9] Chugani, Vinod. "What Is Cosine Distance?" Datacamp.com, DataCamp, 28 July 2024, datacamp.com/tutorial/cosine-distance.

[10] Cisco Systems, Inc. PhishTank: Join the fight against phishing. phishtank.com, 2026.

[11] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS), 2019.